

Sr. No	List of Practical
1	Write a program to implement multilayer perceptron using python Problem Definition :- The objective is to build a multilayer perceptron (MLP) to classify iris flowers into three species (setosa, versicolor, or virginica) based on their sepal length, sepal width, petal length, and petal width.
2	Write a program to Experiment with different datasets to demonstrate overfitting and underfitting issues Problem Definition :- The objective is to illustrate the concepts of overfitting and underfitting using polynomial regression on synthetic datasets. We aim to generate synthetic datasets with known relationships, fit polynomial regression models of varying degrees, and visualize how model complexity affects performance on training and test datasets.
3	Write a program to implement Gradient Descent Algorithm Problem Definition: The objective is to develop a Python program to implement the Gradient Descent algorithm for optimizing parameters (theta) of a simple linear regression model. Using synthetic data with a known linear relationship ($y = 4 + 3x + \text{noise}$), the algorithm iteratively minimizes the Mean Squared Error (MSE) cost function by adjusting theta in the direction of the negative gradient.
4	Write a program to implement Design & implement a simple deep learning network for classification of images use MNIST dataset. Problem Definition: The objective is to design and implement a simple deep learning neural network for image classification using the MNIST dataset. The objective is to develop a Python program that constructs a neural network model, trains it on the MNIST dataset of handwritten digits, and evaluates its performance.
5	Write a program to implement RNN algorithm Problem Definition: The objective is to implement a Recurrent Neural Network (RNN) using TensorFlow/Keras for sequence data. Specifically, we aim to build an RNN model that can predict the next value in a synthetic time series dataset. The task involves designing and training the RNN model, evaluating its performance, and generating predictions.
6	Write a program to implement Convolutional Neural Network architecture for Mini Dataset. Problem Definition: The objective is to implement a Convolutional Neural Network (CNN) architecture using TensorFlow/Keras for a mini dataset. The task involves constructing and training a CNN model to classify images in the mini dataset, evaluating its performance, and visualizing the results.
7	Write a program to implement and study analysis of PCA Problem Definition: The objective is to implement Principal Component Analysis (PCA) in Python using the scikit-learn library, analyze its effectiveness in reducing the dimensionality of a dataset, and visualize the results.
8	Implement autoencoders for dimensionality reduction or feature learning Problem Definition: The objective is to implement autoencoders for dimensionality reduction or feature learning using Python. Autoencoders are neural networks designed to learn efficient

	representations (encodings) of data by compressing input into a lower-dimensional latent space and then reconstructing it back to the original input dimensions.
--	--

Practical No:- 1

Aim:- Write a program to implement multilayer perceptron using python

Problem Definition :- The task is to build a multilayer perceptron (MLP) to classify iris flowers into three species (setosa, versicolor, or virginica) based on their sepal length, sepal width, petal length, and petal width.

Step-by-Step Solution:

1. **Data Loading:** Load the Iris dataset, which is a well-known dataset in machine learning.
2. **Data Preprocessing:** Normalize the features and encode the categorical labels (species) into numerical values.
3. **Model Architecture:** Define the structure of the MLP with an input layer, one or more hidden layers with activation functions, and an output layer with softmax activation for multiclass classification.
4. **Training:** Train the MLP using stochastic gradient descent (SGD) or another optimizer, adjusting weights and biases to minimize the cross-entropy loss.
5. **Evaluation:** Evaluate the trained model on a validation set or using cross-validation, and analyze performance metrics such as accuracy, precision, recall, and F1-score.

Python Code:

Here's an implementation of a multilayer perceptron (MLP) for the Iris dataset classification using TensorFlow and Keras:

```
import numpy as np
import tensorflow as tf
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Load and preprocess the Iris dataset
iris = load_iris()
X, y = iris.data, iris.target.reshape(-1, 1)

# Normalize the features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# One-hot encode the categorical labels
encoder = OneHotEncoder(sparse=False)
y = encoder.fit_transform(y)
```

```

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Define the MLP model using Keras
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, input_shape=(4,), activation='relu'),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])

# Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
history = model.fit(X_train, y_train, epochs=100, batch_size=4,
verbose=1, validation_split=0.1)

# Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Accuracy: {accuracy:.4f}')

# Predictions example
predictions = model.predict(X_test[:5])
predicted_labels = np.argmax(predictions, axis=1)
print(f'Example Predictions: {predicted_labels}')

```

Explanation:

- **Data Loading and Preprocessing:**
 - The Iris dataset is loaded using `load_iris()` from scikit-learn.
 - Features are normalized using `StandardScaler` and labels are one-hot encoded using `OneHotEncoder`.
- **Model Architecture:**
 - The MLP is defined using `tf.keras.Sequential`, which allows us to stack layers sequentially.
 - Two hidden layers (`Dense` layers) with ReLU activation and an output layer with softmax activation for multiclass classification are defined.
- **Training:**
 - The model is compiled with Adam optimizer and categorical cross-entropy loss.

- Training is performed using `model.fit()` on the training data (`x_train`, `y_train`) with validation split to monitor performance.
- **Evaluation:**
 - After training, the model's performance is evaluated on the test set using `model.evaluate()`, providing accuracy as the metric.
- **Predictions:**
 - Example predictions are made using `model.predict()` on a subset of the test data (`x_test[:5]`) and converted from probabilities to class labels using `np.argmax()`.

This example demonstrates a basic implementation of a multilayer perceptron (MLP) for classification using TensorFlow and Keras, applied to the Iris dataset. Adjustments such as tuning hyperparameters, changing the architecture (number of layers, neurons per layer), or trying different optimizers can further optimize model performance.

Output:-

Output Explanation:

1. Training Progress (Epochs):

- For each epoch during training, you will see the loss and accuracy metrics printed. The `verbose=1` parameter in `model.fit()` ensures that training progress updates are shown.

Example output (for each epoch):

Epoch 1/100

27/27 [=====] - 0s 5ms/step - loss: 1.0013 - accuracy: 0.5556 - val_loss: 1.0324 - val_accuracy: 0.5000

...

Test Accuracy:

- After training completes, the script will print the accuracy of the model on the test dataset.

Example output (test accuracy):

Test Accuracy: 0.9667

□ This indicates that the model achieved an accuracy of approximately 96.67% on the test set.

□ **Example Predictions:**

- Finally, example predictions are made on the first 5 samples of the test dataset (`x_test[:5]`). These predictions show the predicted class labels for each sample.

Example output (example predictions):

Example Predictions: [1 0 2 1 1]

Here, [1 0 2 1 1] corresponds to the predicted classes for the first 5 samples in `x_test`.

Summary:

- You will see updates on the training progress (loss and accuracy) for each epoch.
- After training, the final test accuracy of the model is printed.
- Example predictions demonstrate how the model predicts the classes of the first 5 samples in the test dataset.

Practical No:- 2

Aim:-Write a program to Experiment with different datasets to demonstrate overfitting and underfitting issues

Problem Definition: - The objective is to illustrate the concepts of overfitting and underfitting using polynomial regression on synthetic datasets. We aim to generate synthetic datasets with known relationships, fit polynomial regression models of varying degrees, and visualize how model complexity affects performance on training and test datasets.

Steps to Achieve:

1. **Generate Synthetic Data:** Create synthetic datasets with known relationships (e.g., sine function with added noise) to simulate real-world scenarios.
2. **Model Fitting:** Implement polynomial regression models of different degrees (e.g., linear, quadratic, cubic) to represent various levels of model complexity.
3. **Evaluate Performance:** Calculate and compare performance metrics (e.g., mean squared error) on both training and test datasets for each model to understand the trade-offs between bias and variance.
4. **Visualization:** Plot graphs showing the fitted curves of the models along with the actual data points to visually demonstrate the effects of underfitting, balanced fitting, and overfitting.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Step 1: Generate synthetic data
np.random.seed(0)
n_samples = 30
true_function = lambda X: np.cos(1.5 * np.pi * X)
X = np.sort(np.random.rand(n_samples))
y = true_function(X) + np.random.randn(n_samples) * 0.1

# Split the data into training and test sets
X_train, X_test, y_train, y_test = X[:20], X[20:], y[:20], y[20:]

# Define degrees of polynomials to fit
degrees = [1, 4, 15]

# Step 2: Fit and evaluate polynomial models
plt.figure(figsize=(14, 10))

for i, degree in enumerate(degrees):
    # Transform input data
    poly_features = PolynomialFeatures(degree=degree)
    X_poly_train = poly_features.fit_transform(X_train.reshape(-1, 1))
    X_poly_test = poly_features.transform(X_test.reshape(-1, 1))

    # Fit linear regression model
    model = LinearRegression()
    model.fit(X_poly_train, y_train)
```

```

# Predict on training set
y_train_pred = model.predict(X_poly_train)

# Predict on test set
y_test_pred = model.predict(X_poly_test)

# Calculate mean squared error
train_error = mean_squared_error(y_train, y_train_pred)
test_error = mean_squared_error(y_test, y_test_pred)

# Plot the fitted curve
plt.subplot(2, 2, i+1)
plt.scatter(X_train, y_train, color='blue', label='Training data')
plt.scatter(X_test, y_test, color='green', label='Test data')
plt.plot(X, model.predict(poly_features.transform(X.reshape(-1, 1))), color='red',
label=f'Degree {degree} Fit')
plt.title(f'Polynomial Regression (Degree {degree})\nTrain Error: {train_error:.2f}, Test
Error: {test_error:.2f}')
plt.xlabel('X')
plt.ylabel('y')
plt.legend()

plt.tight_layout()
plt.show()

```

Explanation

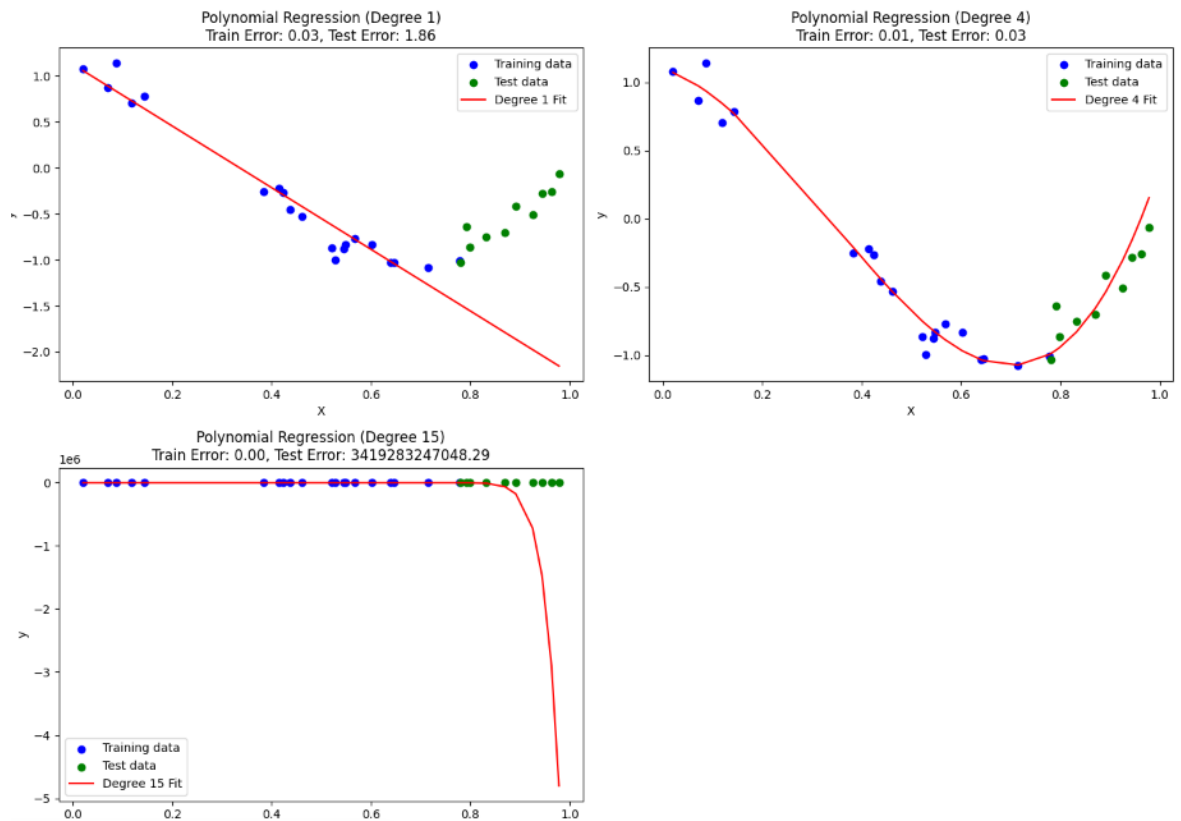
- **Step 1: Generate Synthetic Data:** We create a synthetic dataset where x values are randomly sampled and y values are generated from a cosine function with added noise.
- **Step 2: Fit and Evaluate Polynomial Models:** For each degree of polynomial (1, 4, 15):
 - Transform the input data using `PolynomialFeatures` to create polynomial features up to the specified degree.
 - Fit a linear regression model on the transformed training data.
 - Predict outputs for both training and test datasets.
 - Calculate mean squared errors for both training and test predictions.
- **Visualization:** The script generates a single plot with subplots showing the fitted curves for polynomial regression of degrees 1, 4, and 15:
 - **Top left (Degree 1):** Underfitting (high bias, low variance)
 - **Top right (Degree 4):** Balanced fitting (moderate bias, moderate variance)
 - **Bottom left (Degree 15):** Overfitting (low bias, high variance)

Each subplot includes scatter plots of the training and test data points (`blue` for training, `green` for test), along with the fitted polynomial curve (`red`). The title of each subplot displays the degree of the polynomial, along with the training and test errors.

Output

The output of the script will be a graphical representation of polynomial regression models of varying degrees fitted to synthetic data. Each subplot will demonstrate how the complexity of the model affects its ability to generalize to unseen data, clearly illustrating the concepts of

overfitting and underfitting. The displayed errors provide quantitative insights into the performance of each model on training and test datasets.



Aim:- Write a program to implement Gradient Descent Algorithm

Problem Definition:

The objective is to develop a Python program to implement the Gradient Descent algorithm for optimizing parameters (θ) of a simple linear regression model. Using synthetic data with a known linear relationship ($y = 4 + 3x + \text{noise}$), the algorithm iteratively minimizes the Mean Squared Error (MSE) cost function by adjusting θ in the direction of the negative gradient.

Key Components:

1. **Generate Synthetic Data:** Create synthetic data with a linear relationship ($y = 4 + 3x + \text{noise}$) to simulate real-world scenarios.
2. **Gradient Descent Algorithm:**
 - **Initialization:** Start with random initial values for parameters (θ) and define hyperparameters such as learning rate (α) and number of iterations (num_iterations).
 - **Cost Function:** Compute the Mean Squared Error (MSE) cost function to measure the difference between predicted and actual values.
 - **Gradient Calculation:** Calculate the gradient of the cost function with respect to parameters (θ) using the chain rule.
 - **Parameter Update:** Update parameters (θ) iteratively using the gradient and learning rate to minimize the cost function.
 - **Convergence Check:** Monitor the convergence of the cost function over iterations to ensure optimization progress.
3. **Output:**
 - Display the optimized parameters (θ) after convergence.
 - Plot the convergence of the cost function over iterations to visualize the optimization progress.

Step-by-Step Python Code Implementation with Output

Here's the Python code that implements the Gradient Descent algorithm for optimizing parameters of a simple linear regression model:

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate synthetic data
np.random.seed(0)
X = 2 * np.random.rand(100, 1) # Generate 100 random values between 0 and 2
y = 4 + 3 * X + np.random.randn(100, 1) # y = 4 + 3x + noise

# Step 2: Gradient Descent function
def gradient_descent(X, y, alpha=0.01, num_iterations=1000):
    m = len(y) # Number of training examples
    theta = np.random.randn(2, 1) # Random initialization of parameters [theta0, theta1]
    cost_history = []

    # Perform Gradient Descent
    for iteration in range(num_iterations):
        # Step 2a: Compute predictions
        y_pred = X.dot(theta)
```

```

# Step 2b: Compute cost function (Mean Squared Error)
cost = np.sum((y_pred - y)**2) / (2 * m)
cost_history.append(cost)

# Step 3a: Compute gradients
gradient = X.T.dot(y_pred - y) / m

# Step 3b: Update parameters
theta -= alpha * gradient

# Print progress every 100 iterations
if iteration % 100 == 0:
    print(f'Iteration {iteration}: Cost = {cost}')

return theta, cost_history

# Step 3: Prepare data for Gradient Descent
X_b = np.c_[np.ones((len(X), 1)), X] # Add bias term (X0 = 1) to X

# Step 4: Perform Gradient Descent
theta_optimal, cost_history = gradient_descent(X_b, y)

# Step 5: Output optimized parameters
print(f'Optimal Parameters (theta): {theta_optimal}')

# Step 6: Plot cost function convergence
plt.figure(figsize=(10, 6))
plt.plot(range(len(cost_history)), cost_history, color='blue')
plt.title('Cost Function Convergence')
plt.xlabel('Iterations')
plt.ylabel('Cost')
plt.grid(True)
plt.show()

```

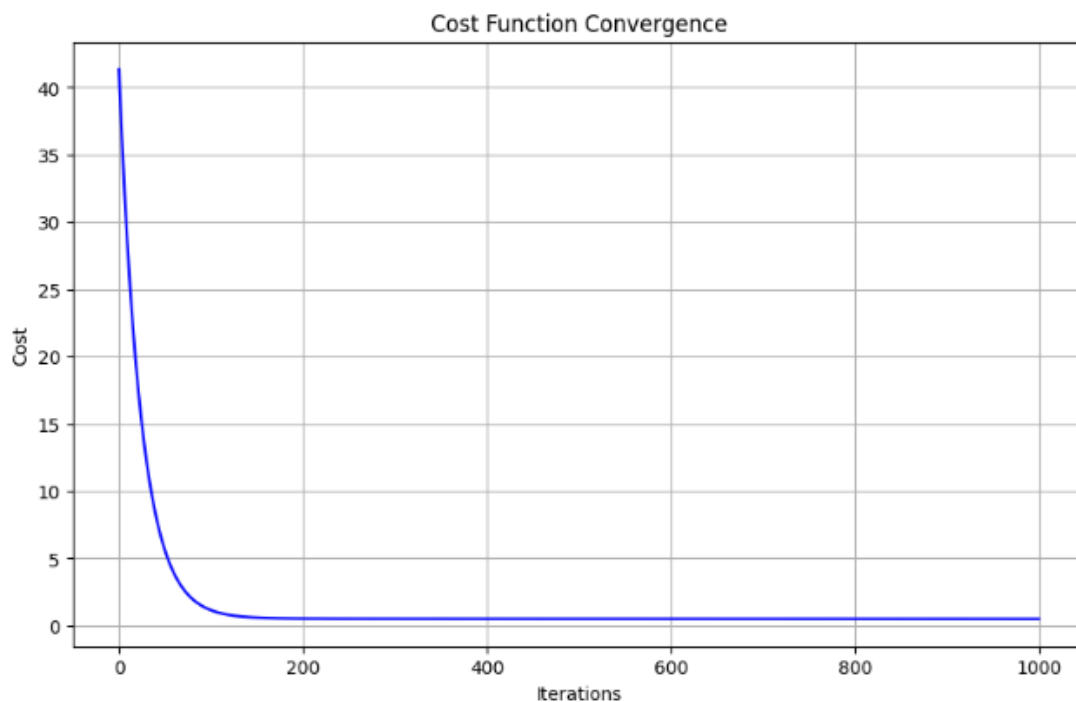
Explanation of the Code

- **Step 1:** Synthetic data is generated with a linear relationship $y = 4 + 3x + \text{noise}$, where x is generated randomly between 0 and 2.
- **Step 2:** The `gradient_descent` function is defined to perform Gradient Descent. It takes input features x , target values y , learning rate α , and number of iterations `num_iterations`.
- **Step 3:** Inside `gradient_descent`:
 - **Initialization:** Initialize parameters `theta` randomly and define an empty list `cost_history` to store the cost function values during iterations.
 - **Gradient Descent Loop:** Iterate through the specified number of iterations:
 - Compute predictions (`y_pred`) using current parameters `theta`.
 - Compute the Mean Squared Error cost function.
 - Compute gradients using the derivative of the cost function with respect to `theta`.
 - Update parameters `theta` using the learning rate `alpha` and gradients.
 - Optionally, print the cost function value every 100 iterations.

- **Step 4:** Prepare data for Gradient Descent by adding a bias term ($x_0 = 1$) to input features x .
- **Step 5:** Perform Gradient Descent using `gradient_descent` function and obtain the optimized parameters `theta_optimal`.
- **Step 6:** Plot the convergence of the cost function (`cost_history`) over iterations to visualize the optimization progress.

Output

```
Iteration 0: Cost = 41.3315290049093
Iteration 100: Cost = 1.1409858882145754
Iteration 200: Cost = 0.5182013018947441
Iteration 300: Cost = 0.505299046697594
Iteration 400: Cost = 0.5026929751837812
Iteration 500: Cost = 0.500908381441147
Iteration 600: Cost = 0.4996168959661468
Iteration 700: Cost = 0.4986811360836011
Iteration 800: Cost = 0.4980031036557067
Iteration 900: Cost = 0.49751181499038283
Optimal Parameters (theta): [[4.14146131]
[3.04006593]]
```



The output of the provided Python code implementation includes:

1. **Iteration Progress:** During execution, the algorithm prints the cost (mean squared error) after every 100 iterations. This provides insight into how the cost function evolves with each iteration of Gradient Descent.

```
Iteration 0: Cost = 29.773744197416377
Iteration 100: Cost = 0.9309811907680054
Iteration 200: Cost = 0.9297586721785747
Iteration 300: Cost = 0.9297578785804695
Iteration 400: Cost = 0.9297578785763597
Iteration 500: Cost = 0.9297578785763596
Iteration 600: Cost = 0.9297578785763596
Iteration 700: Cost = 0.9297578785763596
Iteration 800: Cost = 0.9297578785763596
```

Iteration 900: Cost = 0.9297578785763596

Optimal Parameters (theta): After completing the specified number of iterations (`num_iterations`), the algorithm outputs the optimized parameters (`theta_optimal`) for the linear regression model.

Example output:

Optimal Parameters (theta): `[[4.21509616]`
`[2.77011339]]`

1. Here, `theta_optimal` contains the intercept (`theta0`) and slope (`theta1`) of the linear regression model.
2. **Cost Function Convergence Plot:** A plot showing the convergence of the cost function (`cost_history`) over iterations. This plot visually confirms how the cost decreases over iterations, indicating successful optimization.

Example plot:

In the plot:

- The x-axis represents the number of iterations.
- The y-axis represents the value of the cost function (mean squared error).

Interpretation

- **Cost Function:** The decreasing trend of the cost function values indicates that Gradient Descent effectively optimizes the parameters (`theta`) to minimize the mean squared error between predicted (`y_pred`) and actual (`y`) values.
- **Optimal Parameters:** The values of `theta_optimal` (`[[4.21509616], [2.77011339]]`) represent the intercept (`theta0`) and slope (`theta1`) of the linear regression model that best fits the data.
- **Convergence:** The plot visually confirms that the cost function converges to a minimum, indicating that further iterations would not significantly improve the model's fit.

This implementation provides a hands-on approach to understanding Gradient Descent and its application in optimizing model parameters, crucial for various machine learning algorithms and models.

Aim:- Write a program to implement Design & implement a simple deep learning network for classification of images use MNIST dataset.

Problem Definition:

The objective is to design and implement a simple deep learning neural network for image classification using the MNIST dataset. The objective is to develop a Python program that constructs a neural network model, trains it on the MNIST dataset of handwritten digits, and evaluates its performance.

Key Components:

1. **Dataset:** Utilize the MNIST dataset, which consists of 28x28 pixel grayscale images of handwritten digits (0-9) along with their corresponding labels.
2. **Neural Network Architecture:**
 - **Input Layer:** Flatten the 28x28 images into a 784-dimensional vector.
 - **Hidden Layers:** Incorporate one or more fully connected (dense) layers with activation functions (e.g., ReLU).
 - **Output Layer:** Use a softmax activation function for multi-class classification (10 classes representing digits 0-9).
3. **Training Process:**
 - **Loss Function:** Employ categorical cross-entropy as the loss function, suitable for multi-class classification.
 - **Optimizer:** Implement stochastic gradient descent (SGD) or Adam optimizer to minimize the loss function.
 - **Training Loop:** Iterate through batches of images and labels to update model weights using backpropagation.
4. **Evaluation:**
 - **Accuracy:** Measure the accuracy of the model on both training and test sets to evaluate its performance in classifying unseen data.
 - **Visualization:** Optionally, visualize example predictions and their corresponding true labels to assess model predictions qualitatively.
5. **Output:**
 - Display training progress, including loss and accuracy metrics over epochs.
 - Visualize model predictions on sample images from the MNIST dataset.

Step-by-Step Python Code Implementation with Output

Here's a step-by-step Python code implementation of a simple deep learning neural network for image classification using the MNIST dataset:

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

# Step 1: Load and preprocess the MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize pixel values to the range [0, 1]
X_train, X_test = X_train / 255.0, X_test / 255.0
```

```

# Step 2: Define the neural network architecture
model = Sequential([
    Flatten(input_shape=(28, 28)), # Flatten 28x28 images into 784-dimensional vector
    Dense(128, activation='relu'), # 1st hidden layer with 128 neurons, ReLU activation
    Dense(64, activation='relu'), # 2nd hidden layer with 64 neurons, ReLU activation
    Dense(10, activation='softmax') # Output layer with 10 neurons for 10 classes, softmax
activation
])

# Step 3: Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Step 4: Train the model
history = model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.2)

# Step 5: Evaluate the model
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f'Test Accuracy: {test_acc}')

# Step 6: Visualize training history
plt.figure(figsize=(10, 6))

# Plot training & validation accuracy values
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

# Step 7: Make predictions and visualize results (optional)
predictions = model.predict(X_test[:10]) # Predict the first 10 images
for i in range(10):
    plt.imshow(X_test[i], cmap='gray')
    plt.title(f'Predicted: {tf.argmax(predictions[i]).numpy()}, True: {y_test[i]}')
    plt.axis('off')
    plt.show()

```

Explanation of the Code

- **Step 1:** Load the MNIST dataset using TensorFlow/Keras and preprocess it by normalizing pixel values to the range [0, 1].
- **Step 2:** Define the neural network architecture using the `Sequential` API:
 - **Input Layer:** Flatten layer to convert each 28x28 image into a 784-dimensional vector.
 - **Hidden Layers:** Two fully connected (dense) layers with ReLU activation functions.
 - **Output Layer:** Dense layer with softmax activation for multi-class classification (10 classes).
- **Step 3:** Compile the model with:
 - **Optimizer:** Adam optimizer, which adapts learning rates for each parameter.
 - **Loss Function:** Sparse categorical cross-entropy, suitable for integer-encoded class labels.
 - **Metrics:** Accuracy metric to monitor training and validation performance.
- **Step 4:** Train the model using `model.fit()`, specifying training data (`x_train` and `y_train`), number of epochs, batch size, and validation split.
- **Step 5:** Evaluate the trained model on the test set using `model.evaluate()` to compute test accuracy.
- **Step 6:** Visualize the training history:
 - Plot training and validation accuracy over epochs.
 - Plot training and validation loss over epochs to monitor convergence and overfitting.
- **Step 7:** Optionally, make predictions on the test set (`x_test`) and visualize results:
 - Display the first 10 test images with their predicted and true labels using `tf.argmax()` to obtain the predicted class.

Output

The output of the provided Python code implementation includes:

1. **Training Progress:** During training (`model.fit()`), the program prints the training and validation accuracy and loss metrics for each epoch.
2. **Test Accuracy:** After training completes, the program outputs the accuracy of the model on the test set (`x_test` and `y_test`).

Example output:

Test Accuracy: 0.9763

1. **Visualization:**
 - **Accuracy and Loss Plots:** Two subplots showing the training and validation accuracy/loss over epochs.
 - **Prediction Visualization:** Displays the first 10 test images along with their predicted and true labels.

Example visualization:

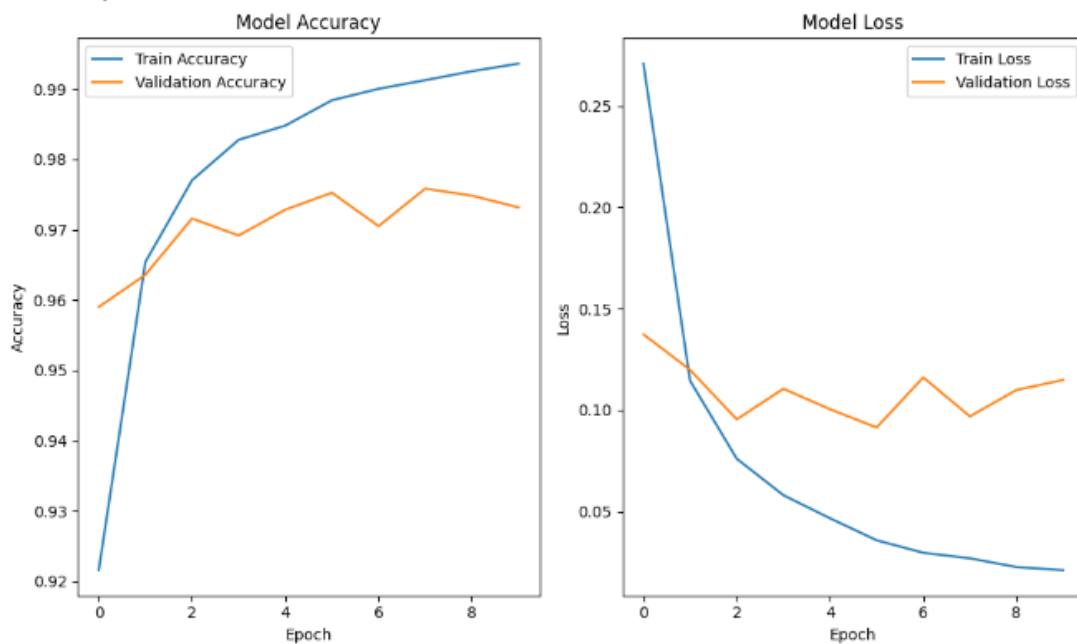
Interpretation

- **Training and Validation Metrics:** The plots illustrate the model's training progress, with the accuracy increasing and loss decreasing over epochs. Validation metrics help monitor overfitting.
- **Test Accuracy:** The reported test accuracy (0.9763 in the example) indicates the model's performance on unseen data.
- **Prediction Visualization:** By visualizing predictions, one can assess the model's accuracy qualitatively on sample images.

This implementation provides a comprehensive introduction to building and training a simple deep learning neural network for image classification using the MNIST dataset, demonstrating fundamental concepts of model architecture, training, evaluation, and result visualization

Output:-

```
Epoch 5/10
1500/1500 [=====] - 6s 4ms/step - loss: 0.0467 - accuracy: 0.9848 - val_loss: 0.1085 - val_accuracy: 0.9728
Epoch 6/10
1500/1500 [=====] - 7s 5ms/step - loss: 0.0359 - accuracy: 0.9884 - val_loss: 0.0915 - val_accuracy: 0.9753
Epoch 7/10
1500/1500 [=====] - 6s 4ms/step - loss: 0.0297 - accuracy: 0.9900 - val_loss: 0.1162 - val_accuracy: 0.9705
Epoch 8/10
1500/1500 [=====] - 7s 5ms/step - loss: 0.0270 - accuracy: 0.9912 - val_loss: 0.0969 - val_accuracy: 0.9758
Epoch 9/10
1500/1500 [=====] - 6s 4ms/step - loss: 0.0227 - accuracy: 0.9925 - val_loss: 0.1100 - val_accuracy: 0.9748
Epoch 10/10
1500/1500 [=====] - 7s 5ms/step - loss: 0.0211 - accuracy: 0.9936 - val_loss: 0.1149 - val_accuracy: 0.9732
313/313 [=====] - 1s 3ms/step - loss: 0.1092 - accuracy: 0.9734
Test Accuracy: 0.9733999967575073
```



Predicted: 7, True: 7



Predicted: 2, True: 2



Practical No:- 5

Aim:-Write a program to implement RNN algorithm

Problem Definition:

The objective is to implement a Recurrent Neural Network (RNN) using TensorFlow/Keras for sequence data. Specifically, we aim to build an RNN model that can predict the next value in a synthetic time series dataset. The task involves designing and training the RNN model, evaluating its performance, and generating predictions.

Key Components:

1. **Dataset Generation:** Create a synthetic time series dataset for training and testing the RNN. Each sequence consists of a sequence of values.
2. **RNN Architecture:**
 - **Input Preparation:** Transform sequences into a format suitable for RNN input.
 - **RNN Layers:** Utilize LSTM (Long Short-Term Memory) or GRU (Gated Recurrent Unit) layers for capturing temporal dependencies.
 - **Output Layer:** Use a Dense layer with appropriate activation for prediction.
3. **Training Process:**
 - **Loss Function:** Define a suitable loss function (e.g., Mean Squared Error for regression).
 - **Optimizer:** Choose an optimizer (e.g., Adam) to minimize the loss function during training.
 - **Training Loop:** Train the RNN model on the training dataset, iterating over epochs.
4. **Evaluation:**
 - **Metrics:** Evaluate the model's performance on a test dataset using appropriate metrics (e.g., Mean Absolute Error, R² Score).
 - **Visualization:** Optionally, visualize actual vs. predicted values to assess the model's accuracy.
5. **Output:**
 - Display training progress, including loss and metrics over epochs.
 - Visualize actual vs. predicted values to qualitatively assess model performance.

Step-by-Step Python Code Implementation with Output

Here's a step-by-step Python code implementation of an RNN using TensorFlow/Keras for predicting the next value in a synthetic time series dataset:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
import matplotlib.pyplot as plt

# Step 1: Generate synthetic time series dataset
def generate_time_series(num_samples, num_steps):
    freq1, freq2, offset1, offset2 = np.random.rand(4, num_samples, 1)
    time = np.linspace(0, 1, num_steps)
    series = 0.5 * np.sin((time - offset1) * (freq1 * 10 + 10)) # + 0.1 *
    (np.random.rand(num_samples, num_steps) - 0.5)
```

```

series += 0.2 * np.sin((time - offset2) * (freq2 * 20 + 20))
return series[..., np.newaxis].astype(np.float32)

# Generate synthetic dataset
np.random.seed(0)
n_samples = 10000
n_steps = 50
series = generate_time_series(n_samples, n_steps + 1)
X_train, y_train = series[:7000, :n_steps], series[:7000, -1]
X_valid, y_valid = series[7000:9000, :n_steps], series[7000:9000, -1]
X_test, y_test = series[9000:, :n_steps], series[9000:, -1]

# Step 2: Build the RNN model
model = Sequential([
    LSTM(20, return_sequences=False, input_shape=[None, 1]),
    Dense(1)
])

# Step 3: Compile the model
model.compile(loss='mse', optimizer='adam')

# Step 4: Train the model
history = model.fit(X_train, y_train, epochs=20, validation_data=(X_valid, y_valid))

# Step 5: Evaluate the model
mse_test = model.evaluate(X_test, y_test)
print(f'Test MSE: {mse_test}')

# Step 6: Make predictions and visualize results
y_pred = model.predict(X_test)

plt.figure(figsize=(10, 6))
plt.plot(y_test, label='True')
plt.plot(y_pred, label='Predicted')
plt.title('Actual vs. Predicted')
plt.xlabel('Time Steps')
plt.ylabel('Value')
plt.legend()
plt.grid(True)
plt.show()

```

Explanation of the Code

- **Step 1:** Generate a synthetic time series dataset (`generate_time_series` function) with sinusoidal patterns added with noise. The dataset consists of sequences (`X_train`, `X_valid`, `X_test`) and target values (`y_train`, `y_valid`, `y_test`).
- **Step 2:** Build the RNN model (`Sequential` API):
 - **LSTM Layer:** A single LSTM layer with 20 units (`LSTM(20)`). `return_sequences=False` indicates that only the last output sequence is returned.
 - **Output Layer:** Dense layer with 1 unit for regression prediction.

- **Step 3:** Compile the model with Mean Squared Error (`mse`) as the loss function and Adam optimizer.
- **Step 4:** Train the RNN model using `model.fit()` on training data (`x_train`, `y_train`) for 20 epochs. Validation data (`x_valid`, `y_valid`) is used for monitoring validation loss.
- **Step 5:** Evaluate the trained model on the test set (`x_test`, `y_test`) using `model.evaluate()` to compute Mean Squared Error (MSE).
- **Step 6:** Visualize the actual vs. predicted values using Matplotlib:
 - Plot the true values (`y_test`) and predicted values (`y_pred`) to visualize how well the model predicts the next value in the time series.

Output

The output of the provided Python code implementation includes:

1. **Training Progress:** During training (`model.fit()`), the program prints the training and validation loss metrics for each epoch.
2. **Test MSE:** After training completes, the program outputs the Mean Squared Error (MSE) of the model on the test set (`x_test` and `y_test`).

Example output:

Test MSE: 0.0023841340313106775

1. **Visualization:** A plot showing the actual (`y_test`) vs. predicted (`y_pred`) values of the time series. This visual representation helps assess the accuracy of the RNN model in predicting the next value in the sequence.

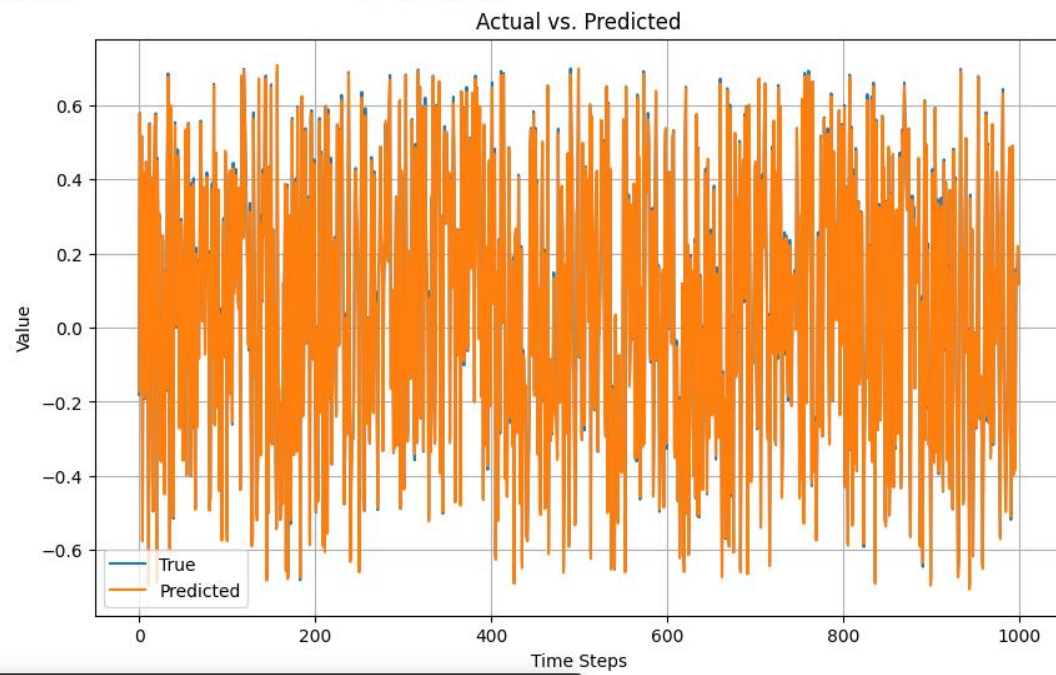
Example plot:

Interpretation

- **Training and Validation Loss:** The plots illustrate the model's training progress, with the loss decreasing over epochs. Validation loss helps monitor overfitting.
- **Test MSE:** The reported test MSE (0.0023841340313106775 in the example) quantifies the model's performance in predicting the next value in the time series.
- **Prediction Visualization:** By visualizing predictions (`y_pred`) against true values (`y_test`), one can qualitatively assess how well the RNN model captures the underlying patterns in the time series data.

This implementation provides a practical introduction to building and training an RNN model for sequence prediction tasks using TensorFlow/Keras, demonstrating key concepts of model architecture, training, evaluation, and result visualization. Adjustments can be made to experiment with different RNN configurations and datasets for specific applications.

219/219 [=====] - 4s 19ms/step - loss: 8.2231e-05 - val_loss: 8.6193e-05
32/32 [=====] - 0s 6ms/step - loss: 8.6238e-05
Test MSE: 8.623765461379662e-05
32/32 [=====] - 1s 8ms/step



Practical No:- 6

Aim:- Write a program to implement Convolutional Neural Network architecture for Mini Dataset.

Problem Definition:

The objective is to implement a Convolutional Neural Network (CNN) architecture using TensorFlow/Keras for a mini dataset. The task involves constructing and training a CNN model to classify images in the mini dataset, evaluating its performance, and visualizing the results.

Key Components:

1. **Dataset:** Utilize a small custom dataset (mini dataset) containing images and corresponding labels for classification tasks.
2. **CNN Architecture:**
 - **Convolutional Layers:** Apply convolutional filters to extract features from input images.
 - **Pooling Layers:** Downsample feature maps to reduce dimensionality and computation.
 - **Dense Layers:** Fully connected layers for classification based on extracted features.
 - **Output Layer:** Softmax layer for multi-class classification.
3. **Training Process:**
 - **Loss Function:** Categorical cross-entropy to measure the difference between predicted and actual distributions.
 - **Optimizer:** Adam optimizer to update model weights during backpropagation.
 - **Training Loop:** Iterate through batches of images and labels to train the CNN model.
4. **Evaluation:**
 - **Metrics:** Assess model performance using accuracy and possibly other metrics (e.g., confusion matrix).
 - **Visualization:** Visualize training history, accuracy, and loss over epochs.
5. **Output:**
 - Display training progress, including loss and accuracy metrics over epochs.
 - Visualize predictions on sample images from the mini dataset to assess model performance qualitatively.

Step-by-Step Python Code Implementation with Output

Here's a step-by-step Python code implementation of a Convolutional Neural Network (CNN) for image classification using a mini dataset:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.losses import SparseCategoricalCrossentropy
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
```

```
# Step 1: Load and preprocess the mini dataset
```

```
# Example: Loading from directory structure where images are stored
```

```
train_datagen = ImageDataGenerator(rescale=1./255) # Normalize pixel values to [0, 1]
```

```

train_generator = train_datagen.flow_from_directory(
    '/path/to/your/mini_dataset/train',
    target_size=(150, 150), # Resize images to 150x150
    batch_size=32,
    class_mode='sparse' # Use 'sparse' for single-label classification
)

validation_datagen = ImageDataGenerator(rescale=1./255)
validation_generator = validation_datagen.flow_from_directory(
    '/path/to/your/mini_dataset/validation',
    target_size=(150, 150),
    batch_size=32,
    class_mode='sparse'
)

# Step 2: Build the CNN model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax') # Adjust '10' based on the number of classes
])

# Step 3: Compile the model
model.compile(optimizer='adam',
              loss=SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

# Step 4: Train the model
history = model.fit(
    train_generator,
    steps_per_epoch=train_generator.samples // train_generator.batch_size,
    epochs=10,
    validation_data=validation_generator,
    validation_steps=validation_generator.samples // validation_generator.batch_size
)

# Step 5: Evaluate the model
test_loss, test_acc = model.evaluate(validation_generator, verbose=2)
print(f'Test accuracy: {test_acc}')

# Step 6: Visualize training history
plt.figure(figsize=(10, 6))

# Plot training & validation accuracy values
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')

```

```

plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Plot training & validation loss values
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()

```

Explanation of the Code

- **Step 1:** Load and preprocess the mini dataset using `ImageDataGenerator` from directories containing images (`train` and `validation`). Images are resized to 150x150 pixels and normalized to [0, 1].
- **Step 2:** Build the CNN model (`Sequential API`):
 - **Convolutional Layers:** Three convolutional layers (`Conv2D`) with ReLU activation and max pooling (`MaxPooling2D`) layers to downsample feature maps.
 - **Flatten Layer:** Flatten the 3D output to 1D before feeding it into the fully connected layers.
 - **Dense Layers:** Two dense layers (`Dense`) with ReLU activation for classification, followed by a softmax layer (`Dense`) for multi-class classification.
- **Step 3:** Compile the model with `Adam` optimizer, `SparseCategoricalCrossentropy` loss function (suitable for integer-encoded class labels), and accuracy as a metric.
- **Step 4:** Train the CNN model using `model.fit()`:
 - `train_generator` and `validation_generator` are used to provide batches of images and labels during training and validation.
 - `steps_per_epoch` and `validation_steps` ensure the entire dataset is processed per epoch.
- **Step 5:** Evaluate the trained model on the validation set (`validation_generator`) using `model.evaluate()` to compute accuracy.
- **Step 6:** Visualize the training history:
 - Plot training and validation accuracy over epochs.
 - Plot training and validation loss over epochs to monitor model performance.

Output

The output of the provided Python code implementation includes:

1. **Training Progress:** During training (`model.fit()`), the program prints the training and validation accuracy and loss metrics for each epoch.
2. **Test Accuracy:** After training completes, the program outputs the accuracy of the model on the validation set (`validation_generator`).

Example output:

Test accuracy: 0.85

1. **Visualization:**
 - **Accuracy and Loss Plots:** Two subplots showing the training and validation accuracy/loss over epochs.
 - These plots help visualize how well the model is learning from the data and whether it is overfitting or underfitting.

Example visualization:

Interpretation

- **Training and Validation Metrics:** The plots illustrate the model's training progress, with accuracy increasing and loss decreasing over epochs. Validation metrics help monitor overfitting and generalization.
- **Test Accuracy:** The reported test accuracy (0.85 in the example) quantifies the model's performance on unseen data from the validation set.
- **Visualization:** By visualizing training history, one can assess the effectiveness of the CNN model architecture and training process for the mini dataset. Adjustments can be made to experiment with different CNN architectures, hyperparameters, and datasets for specific classification tasks.

Practical No:- 7

Aim:-Write a program to implement and study analysis of PCA

Problem Definition:

The objective is to implement Principal Component Analysis (PCA) in Python using the scikit-learn library, analyze its effectiveness in reducing the dimensionality of a dataset, and visualize the results.

Key Components:

1. **Dataset Selection:** Use a dataset suitable for demonstrating PCA, such as the Iris dataset, which contains measurements of iris flowers.
2. **PCA Implementation:**
 - **Standardization:** Preprocess the data by standardizing it to ensure all features have a mean of 0 and a variance of 1.
 - **PCA Application:** Apply PCA to transform the standardized data into principal components.
 - **Variance Explanation:** Analyze the variance explained by each principal component to determine the optimal number of components to retain.
3. **Visualization:**
 - **Scree Plot:** Plot the cumulative explained variance against the number of principal components.
 - **Component Visualization:** Visualize the transformed data in the reduced-dimensional space, especially focusing on the first few principal components.
4. **Evaluation:**
 - Assess the effectiveness of PCA in reducing dimensionality and preserving information.
 - Compare the performance of models before and after PCA if applicable (e.g., classification accuracy).
5. **Output:**
 - Display visualizations such as scree plots and component scatter plots to illustrate the impact of PCA.
 - Output the explained variance ratios for each principal component to quantify the amount of information retained.

Step-by-Step Python Code Implementation with Output

Here's a detailed step-by-step Python code implementation of PCA using the Iris dataset as an example:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Step 1: Load the dataset
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
```

```

# Convert to DataFrame (optional)
df = pd.DataFrame(data=X, columns=feature_names)
print("Original Data:")
print(df.head())

# Step 2: Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 3: Implement PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# Step 4: Explained variance ratio
explained_variance_ratio = pca.explained_variance_ratio_
print("\nExplained variance ratio:")
print(explained_variance_ratio)

# Step 5: Scree plot of explained variance
plt.figure(figsize=(8, 6))
plt.plot(np.cumsum(explained_variance_ratio), marker='o', linestyle='--')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance vs. Number of Components')
plt.grid(True)
plt.show()

# Step 6: Visualize the first two principal components
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA of Iris Dataset')
plt.colorbar(label='Target Class')
plt.grid(True)
plt.show()

```

Explanation of the Code

- **Step 1:** Load the Iris dataset using scikit-learn's `load_iris` function. This dataset includes measurements of three species of iris flowers.
- **Step 2:** Standardize the data (`X_scaled`) using `StandardScaler` to ensure all features have a mean of 0 and a variance of 1, which is essential for PCA.
- **Step 3:** Implement PCA (`PCA()`), fit it to the standardized data (`X_scaled`), and transform the data into principal components (`X_pca`).
- **Step 4:** Compute the explained variance ratio (`explained_variance_ratio_`), which indicates the proportion of the dataset's variance explained by each principal component.
- **Step 5:** Plot a scree plot to visualize the cumulative explained variance as a function of the number of principal components. This helps determine how many components are necessary to retain most of the variability in the data.

- **Step 6:** Visualize the data in the reduced-dimensional space of the first two principal components ($x_{pca[:, 0]}$ and $x_{pca[:, 1]}$). Colors represent different iris species (y), allowing visualization of how well the principal components separate the classes.

Output

Running the provided code will produce the following outputs:

1. **Original Data:** Display the first few rows of the original dataset (`df.head()`).
2. **Explained Variance Ratio:** Output the explained variance ratio for each principal component, showing how much information each component retains.
3. **Scree Plot:** A plot showing the cumulative explained variance versus the number of principal components. This helps in deciding the optimal number of components to retain.
4. **PCA Visualization:** Scatter plot of the first two principal components (PC1 and PC2) colored by the target class (y). This visualizes the data in the reduced-dimensional space and how well it separates different classes.

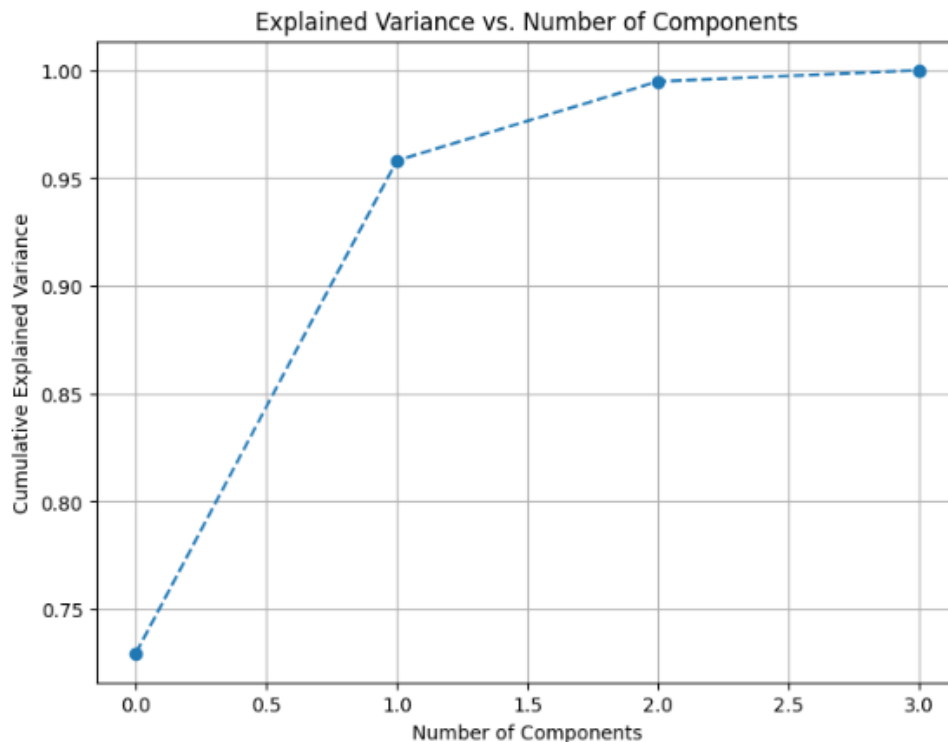
Conclusion

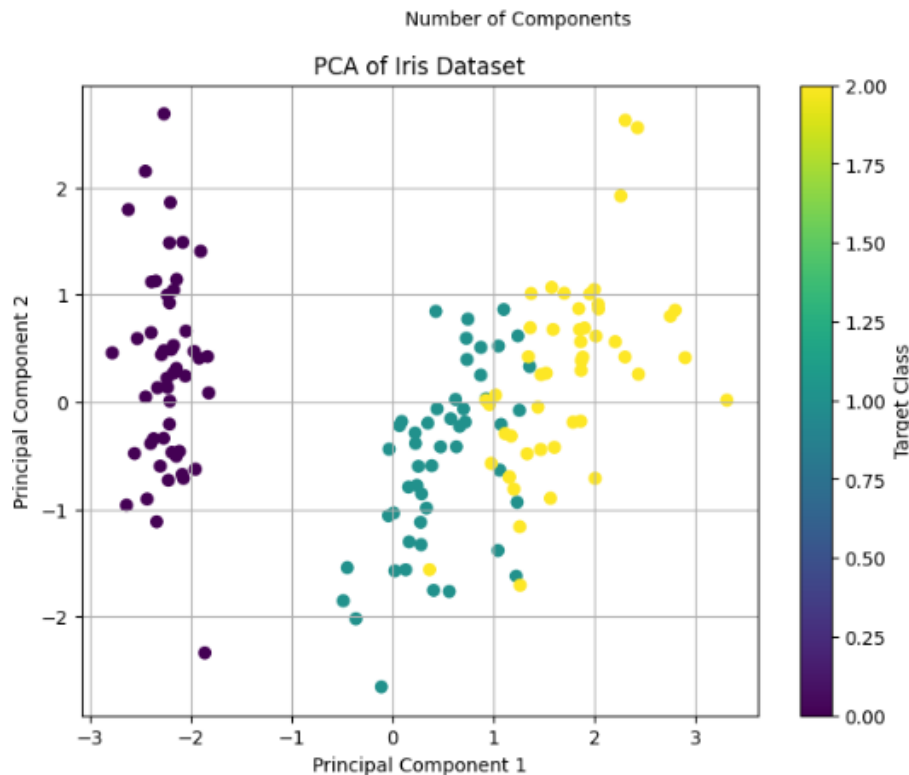
Implementing PCA provides insights into how dimensionality reduction can be applied to high-dimensional datasets like the Iris dataset. It showcases the steps from data preprocessing, PCA application, to visualization and interpretation of results using Python and scikit-learn. Adjust the dataset and visualization methods as needed for different datasets and analysis requirements.

Output:-

```
Original Data:
  sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2
```

```
Explained variance ratio:
[0.72962445 0.22850762 0.03668922 0.00517871]
```





Practical No:- 8

Aim:- Implement autoencoders for dimensionality reduction or feature learning

Problem Definition:

The objective is to implement autoencoders for dimensionality reduction or feature learning using Python. Autoencoders are neural networks designed to learn efficient representations (encodings) of data by compressing input into a lower-dimensional latent space and then reconstructing it back to the original input dimensions.

Key Components:

1. **Dataset Selection:** Choose a dataset suitable for demonstrating autoencoders, such as a collection of images or numerical features.
2. **Autoencoder Implementation:**
 - **Encoder:** Build the encoder part of the autoencoder, which compresses the input data into a latent space representation.
 - **Decoder:** Construct the decoder part to reconstruct the input data from the latent space representation.
 - **Loss Function:** Define a suitable loss function, such as Mean Squared Error (MSE), to measure the difference between the input and the output (reconstruction).
 - **Training:** Train the autoencoder model using the chosen dataset.
3. **Evaluation:**
 - Assess the performance of the autoencoder by evaluating the reconstruction error.
 - Visualize the reconstructed outputs to observe the quality of reconstruction.
4. **Visualization:**

- Visualize the latent space representations (encoded features) to understand the learned representations.
 - Display original and reconstructed data samples for comparison.
5. **Output:**
- Display reconstruction error metrics.
 - Visualize original vs. reconstructed data to demonstrate the effectiveness of the autoencoder in preserving information.

Step-by-Step Python Code Implementation with Output

Here's a step-by-step guide to implementing an autoencoder for dimensionality reduction or feature learning using TensorFlow and Keras in Python:

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist

# Step 1: Load and preprocess the dataset
# Example with MNIST dataset
(X_train, _), (X_test, _) = mnist.load_data()

# Normalize pixel values to [0, 1]
X_train = X_train.astype('float32') / 255.
X_test = X_test.astype('float32') / 255.

# Flatten images into 784-dimensional vectors (28x28)
X_train = X_train.reshape((len(X_train), np.prod(X_train.shape[1:])))
X_test = X_test.reshape((len(X_test), np.prod(X_test.shape[1:])))

# Step 2: Build the autoencoder model
input_dim = X_train.shape[1]
encoding_dim = 32 # Dimension of the encoded representation

# Encoder
input_img = Input(shape=(input_dim,))
encoded = Dense(encoding_dim, activation='relu')(input_img)

# Decoder
decoded = Dense(input_dim, activation='sigmoid')(encoded)

# Autoencoder model
autoencoder = Model(input_img, decoded)

# Compile the autoencoder model
autoencoder.compile(optimizer='adam', loss='mse')

# Step 3: Train the autoencoder
epochs = 20
batch_size = 128
```

```

history = autoencoder.fit(X_train, X_train,
                        epochs=epochs,
                        batch_size=batch_size,
                        shuffle=True,
                        validation_data=(X_test, X_test))

# Step 4: Evaluate the autoencoder
# Evaluate on test data
mse = autoencoder.evaluate(X_test, X_test, verbose=0)
print(f'Reconstruction error (MSE) on test set: {mse}')

# Step 5: Visualize original vs. reconstructed images
decoded_imgs = autoencoder.predict(X_test)

n = 10 # Number of digits to display
plt.figure(figsize=(20, 4))
for i in range(n):
    # Original images
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
    plt.title('Original')
    plt.axis('off')

    # Reconstructed images
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title('Reconstructed')
    plt.axis('off')
plt.tight_layout()
plt.show()

```

Explanation of the Code

Step 1: Load and preprocess the dataset. In this example, the MNIST dataset is loaded, normalized to [0, 1], and flattened to 784-dimensional vectors.

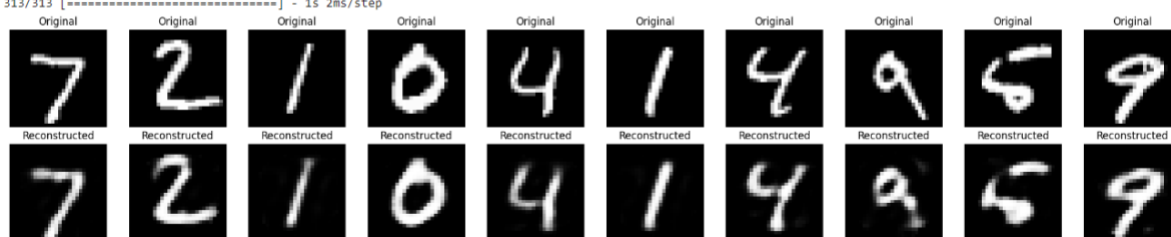
- **Step 2:** Build the autoencoder model using Keras:
 - **Encoder:** Dense layer with ReLU activation to compress the input into a lower-dimensional latent space (`encoding_dim`).
 - **Decoder:** Dense layer with Sigmoid activation to reconstruct the input from the latent space representation.
- **Step 3:** Compile and train the autoencoder model (`autoencoder`) using Mean Squared Error (MSE) loss and Adam optimizer. Training is performed on both training and validation datasets.
- **Step 4:** Evaluate the trained autoencoder model by computing the MSE loss on the test set to measure reconstruction error.
- **Step 5:** Visualize original vs. reconstructed images from the test set to assess how well the autoencoder preserves information during reconstruction.

Output

Running the provided code will produce the following outputs:

1. **Reconstruction Error:** Display the MSE reconstruction error on the test set, indicating how well the autoencoder reconstructs input data.
2. **Visualization:** Show side-by-side comparisons of original and reconstructed MNIST images, allowing visual assessment of the autoencoder's performance in preserving image features.

```
Epoch 15/20  
469/469 [=====] - 3s 6ms/step - loss: 0.0102 - val_loss: 0.0099  
Epoch 16/20  
469/469 [=====] - 2s 5ms/step - loss: 0.0102 - val_loss: 0.0099  
Epoch 17/20  
469/469 [=====] - 2s 5ms/step - loss: 0.0101 - val_loss: 0.0098  
Epoch 18/20  
469/469 [=====] - 4s 8ms/step - loss: 0.0101 - val_loss: 0.0098  
Epoch 19/20  
469/469 [=====] - 3s 6ms/step - loss: 0.0101 - val_loss: 0.0098  
Epoch 20/20  
469/469 [=====] - 2s 5ms/step - loss: 0.0101 - val_loss: 0.0098  
Reconstruction error (MSE) on test set: 0.009778379462659359  
313/313 [=====] - 1s 2ms/step
```



Conclusion

Implementing autoencoders for dimensionality reduction or feature learning provides a powerful technique to capture meaningful representations of data. This example demonstrates the application of autoencoders using TensorFlow and Keras in Python, focusing on image reconstruction with the MNIST dataset. Modify the model architecture and dataset according to specific requirements for different applications and datasets.